Ray Tracing Acceleration using Bounding Volume Hierarchical Structures

Department of Mechanical and Mechatronics Engineering

A Report Prepared For:
The University of Waterloo

Prepared By:
Mckale Chung
268 Phillip Street, 35 Clayfield
Waterloo, ON N2L 6G9
May 5th, 2025

268 Phillip Street, 35 Clayfield
Waterloo, ON N2L 6G9

May 5th, 2025

Dr. Derek Wright
Director, Mechatronics Engineering
Department of Mechanical and Mechatronics Engineering
University of Waterloo
200 University Ave W.
Waterloo, Ontario, N2L 3G1

Dear Dr. Wright,

This report, entitled “Ray Tracing Acceleration using Bounding Volume Hierarchical Structures”, was prepared as my 3A work term report.

This is a self-study report project that explores a software optimization algorithm for ray tracers. It aims to compare this algorithm against a naïve ray tracing solution to reduce the number of computations performed during a render. This was built off a personal ray tracer to improve efficiency.

This report was written entirely by me and has not received any previous academic credit at this or any other institution. My responsibilities for the project included researching the optimization techniques, designing all test scenes, tracking computations, and the implementation of all math and rendering shown in this report.

Best Regards

Mckale Chung
ID: 20990860
3A Mechatronics Engineering

Table of Contents

List of Figures

List of Tables

List of Equations

Executive Summary

This report covers the implementation and analysis of the Bounding Volume Hierarchy Acceleration algorithm, which is used to reduce the average number of intersection computations per ray. This is done by converting a scene into a tree structure by grouping objects in reference to their bounding boxes. A complex 3D scene may contain thousands or millions of primitive objects. For a naïve ray tracer, this would require a number of computations that scales linearly with the number of primitives.

The report covers BVH tree traversal, along with two methods of BVH construction. These two methods were Spatial Median Split (SMS) and Surface Area Heuristic (SAH). Using a pre-order tree traversal strategy, the system culls large regions of space where no intersections occur, leading to significant performance improvements.

Three scenes of increasing complexity (from 200 to 278104 primitives) were tested. In this case, one ray was cast per pixel, with no ray bounces. The SAH method consistently outperformed both SMS and the naïve approach. For example, in the most complex scene, the chess scene, the naïve method performed 278104 intersection tests per ray, while SAH reduced this to 137—a 99.95% reduction. The SMS method provided moderate improvements at 243 intersection tests per ray but generally struggles more than the SAH method for unbalanced scenes. Additionally, the BVH methods reduced rendering time significantly, with the SAH method completing in 51 minutes what the naïve method failed to complete in over 21 days.

The report used a loose mathematical criterion based on logarithmic scaling to compare efficiency independently of hardware. This was chosen on the relative time complexities of general tree-traversal against the naïve case. While the criterion was sufficient for the tested scenes, the report recommends refining it for future work by potentially basing it on the number of primitives in the scene

Key limitations include the use of a single-threaded CPU and fixed BVH parameters. Potential improvements could be exploring GPU-based multi-threading, and more modified splitting strategy for further optimization. While successfully reducing the number of computations, there are significant areas of research and opportunities to further optimize the ray tracer.

1.0 Introduction

Raytracing is a computer graphics technique to simulate how light interacts with objects to render a 3D scene. A scene consists of objects and a virtual camera positioned in 3D space. The aim of rendering is to draw out what the camera sees, as a video or image. It works by sending out "rays" for each pixel location in an image. If that ray intersects with any object, then it will draw the color of that object at that pixel point.

Video games may render scenes in real time, while studios such as Pixar use a ray tracer that takes over x time to render a single frame for maximum quality. Lighting, reflections, refraction, shadows and more can be described mathematically. By performing such calculations whenever a ray intersects with some object in a scene, powerful ray tracers can simulate photo-realistic results.

Basic shapes in graphics are referred to as primitives. These are shapes such as spheres and triangles, where ray-primitive intersection calculations are straight forward. The more primitives there are in a scene, the more intersection comparisons to compute. Complex shapes are normally described as meshes; a collection of primitives. The vast majority of meshes are composed of triangles, and each triangle is described by 3 vertices in space. These vertices describe the x, y, and z coordinate of the point, and every single point comes together to describe an entire mesh. The figure below describes the Utah teapot, a common test rendering model in computer graphics. The simplest version has 3488 triangles. [1]

Figure 1 Utah Teapot, Example of a mesh

1.1 Objective

The aim of this report is to analyze a method of ray tracing optimization called Bounding Volume Hierarchy Acceleration. Given a static ray tracer that renders a single scene, the report explores algorithmic approaches to reduce the number of ray-object computations required, resulting in a render speed-up. The report aims to record the implementation of this optimization method and a comparison against the naïve implementation of ray tracing.

2.0 Problem Definition

As developers implement more features into their ray tracers, the complexity scales, and the rendering time increases. However, since it is vital in entertainment to generate scenes that are both high resolution and look 'visually appealing', these rendering techniques cannot be dropped. Therefore, algorithms are developed to balance the scale and optimize how ray tracers operate. Ideally, a ray only actually needs to intersect with the closest primitive, but the ray tracer is not aware of which that is. Therefore, algorithms attempt to increase efficiency by reducing the number of objects they must compare to.

2.1 Naïve Implementation

When a basic ray tracer starts to render, it will send out a ray, then check if it intersects with every single primitive in the primitive list. Even if the ray does not intersect with any primitive, it still must make a comparison calculation. Even if the ray is completely on the opposite side of the scene, far away from every primitive, the ray tracer will still have to check. Meshes are often comprised of thousands of triangles in the tens or hundreds, and it is inefficient to make that many intersections check for every ray.

It is fair to question why each ray needs to compare against every single primitive. The issue with ray-object intersections is that the ray-tracer is not aware of which object is closest until it checks every single object in the scene. As it moves through objects, it keeps track of the closest intersection, updating if any collisions happen any closer.

2.2 Bounding Volume Hierarchies

A BVH is a tree structure used to optimize the ray-object intersections performed. It groups primitives by their proximity to each other in a 'bounding box' [2]. A bounding box is a box that surrounds the entirety of a single primitive, or a collection of primitives. These can overlap.

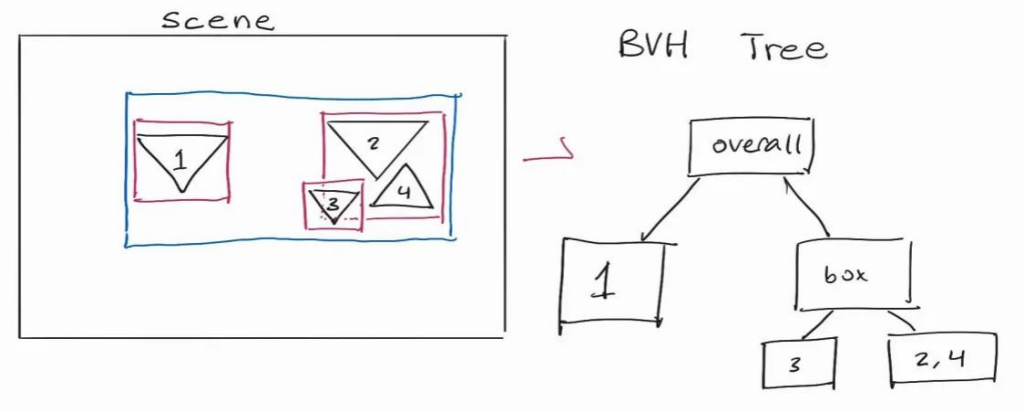

Figure 2. Visualization of Simple BVH Tree Generation

In general tree syntax, the bottom nodes are called leaf nodes. For a BVH structure, the leaf nodes contain a grouping of objects. All other nodes only refer to a bounding box. These can be referred to as box nodes.

2.4 Criteria

Ray tracing and graphics are a constantly evolving field for optimization. There is no full "ideal" measure of optimization, just methods to make little improvements over time. The aim of the report is to generate a faster render. It will use the criteria of computational time.

The naïve method has a time complexity of O(N), where N refers to the size of data. For a ray tracer that would be the number of primitives. This just means that the number of computations grows linearly with the number of primitives. BVH structures and traversing through trees in general have a time complexity of O(log N), based on the log of N with any base [3]. Therefore, the report will consider the optimization as successful as long as the following condition is met:

$$average\ \#\ intersection\ tests\ per\ pixel\ with\ BVH \leq 10log_2 (average\ \#\ intersection\ tests\ per\ pixel\ with\ the\ naive\ implementation) \quad (1)$$

This is to generalize the comparison. All the algorithms should be independent of the system specifications of the computer running the code. Therefore, a decision was made to base the criteria on the number of computations, rather than the time taken to perform the calculations

Since log functions grow slowly as the input is larger, it is not realistic to simply base the criteria on the log function alone (ie. $\log_{10}(1000000000) = 9$). Therefore, a scalar of 10 was added to the

criteria, which is still a significant reduction in complexity compared to the naïve case. The scalar can be chosen relatively arbitrarily, as time complexity only needs to be dependent on the base function (ie log N or N).

2.5 Constraints

The implementation of BVH optimization is based on an existing naïve raytracer by the author. Therefore, there may be limitations in other areas of the code base that are not covered in this report, such as potential sub-optimal calculations for lighting. It is assumed that all comparisons between the naïve and optimized renders will be based on the exact same scenes. There will be no modifications to the code between these aside from the method of choosing ray-object intersections. Therefore, all methods should produce the same images, just with different rendering durations.

This report uses a common assumption that all ray tracing is done on a single-threaded CPU with C++. This means that it will perform all calculations step by step sequentially. Many industry level ray tracers are done by using multithreading, where calculations can be performed simultaneously [4]. These are faster than single-threaded systems in the time taken to run a program, but do not change the total number of ray-object intersection computations. Regardless, the techniques explored in this report sustain the same ‘relative’ speed to each other, regardless of single threading or multi-threading systems. [5]

While arguably the criteria is independent of the computer system used, computers may be inaccurate in floating point roundoff. This could slightly change the way that the bounding boxes are constructed between different computers. However, all tested renders were run on the same computer.

3.0 Bounding Volume Hierarchies Traversal

A BVH is a tree structure used to optimize the ray-object intersections performed. It groups primitives by their proximity to each other in a 'bounding box'. A bounding box is a box that surrounds the entirety of a single primitive, or a collection of primitives. These can overlap. Another important term is the bounding box centroid, which is just the point that is in the very middle of the bounding box.

In general tree syntax, the bottom nodes are called leaf nodes. For a BVH structure, the leaf nodes contain a grouping of objects. All other nodes only refer to a bounding box. These may be referred to as box nodes.

Referring to Figure 3, the node labeled box is just a box node, but it also describes a sub-tree, which child nodes containing 3 and 2 and 4 respectively.

3.1 BVH Traversal Implementation

Since the BVH structure is that of a tree, common computer science tree traversal algorithms can be used. For this project, a pre order version was used, in which the algorithm visits the root node, then the left node, then the right node [6]. This technique was chosen because the bounding box of the root node should contain all bounding boxes of lower nodes. The generalized traversal algorithm is as follows:

1. Start at root
2. Test whether the ray intersects the bounding box of the current node
3. If there is no intersection, stop traversal along this branch.
4. If there is an intersection
    a. If the node is a leaf, test the ray against all primitives contained within the node.
    b. If the node is an internal (box) node, recursively traverse its child nodes by repeating from Step 2.

3.2 Abstract Comparison

BVH structures can be implemented into a ray tracer by replacing the order of ray-object calculations. For example, if the tracer sends a ray into the top-left corner of the scene, it should not intersect with any of the objects. The naïve implementation will have to check if every single ray intersects with all four objects. However, a BVH based method will check if the ray hits the

overall bounding box. If not, then it safely concludes that the ray does not intersect with any objects in the scene. Therefore, it makes one comparison, compared to the four comparisons in the naïve version.

However, in the worst case, it must make more comparisons than the naïve method: one at root, one at the node containing object 1, one at the right child box, then three for the other 2 child nodes.

While the worst case requires more comparisons than the naive, BVH structures are generally more effective as the number of primitives increases. This is because the worst case happens very infrequently. It averages a lower number of comparisons throughout the entire process.

4.0 Implementation of BVH Structures

There are numerous different ways of generating BVH structures. The report explores two in particular:

1. Spatial Median Split
2. Surface Area Heuristic

This report uses the same pre-order method of tree traversal for both of these, so any optimizations between them come from how the BVH tree is structured based on the method.

4.1 Spatial Median Split (SMS)

For both cases, the input is still the list of primitives, and the overall bounding box of all primitives is calculated and available. Recalling that scenes are in 3D space, so the bounding box has 3D dimensions.

SMS first checks which is the longest axis of the overall bounding box, in the x, y, or z axes. It splits the overall bounding box at the middle of this axis. It then repeatedly sorts the objects in the scene depending on where they are compared to this center split [7]. This happens repeatedly, depending on the max number of primitives the developer wants in the leaf nodes.

The figure below describes a simple generated tree using SMS. Assume that the developer specified that leaf nodes may contain a maximum of two primitive objects. The image is flattened to two axes

for simplicity in visualization. It first splits down the middle of the overall bounding box. Since there are three objects on the left side, it splits that side again, generating the corresponding tree.

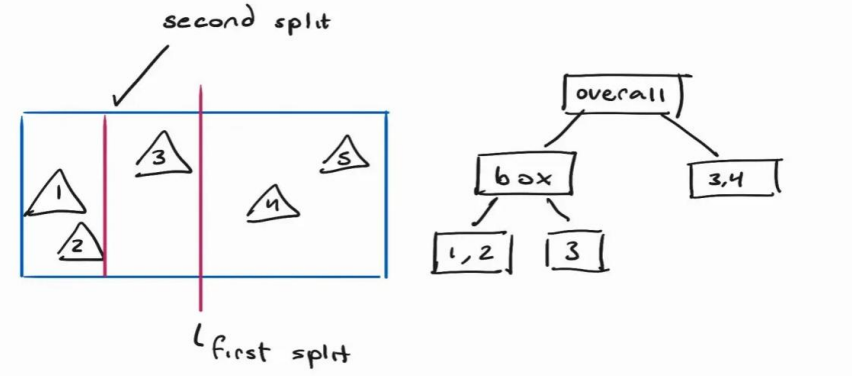

Figure 3. Visualization of Spatial Median Split BVH Tree Generation

SMS does not discriminate based on object distribution. It will always split down the middle, and so it is possible for the primitives to lie unequally on one side of the split. This creates an unbalanced BVH tree, which leads to more required comparisons [8].

4.2 Surface Area Heuristic (SAH) Bounding Volume Hierarchies

SAH is similar to SMS because it splits up primitives based on the bounding boxes. However, it does so in a slightly "smarter" method. As mentioned earlier, if a ray does not intersect with a bounding box, then it does not need to check if it intersects with all the primitives within the bounding box. Since rays are more likely to intersect with large bounding boxes, the intuition behind SAH is to generate sections that minimize the surface area of the grouping [9].

Again, methods will select an axis to operate on. Rather than immediately splitting it, SAH method will use a cost function to estimate the cost of traversing at that point. It essentially says, "If the scene is split here, how efficient will the traversal be". The implementation of this report first proposes eleven equally spaced splits. Depending on the surface areas of the primitives to the left and right sides of the split, the cost function will output a value, and the lowest result determines the most efficient split (based on the eleven proposed splits).

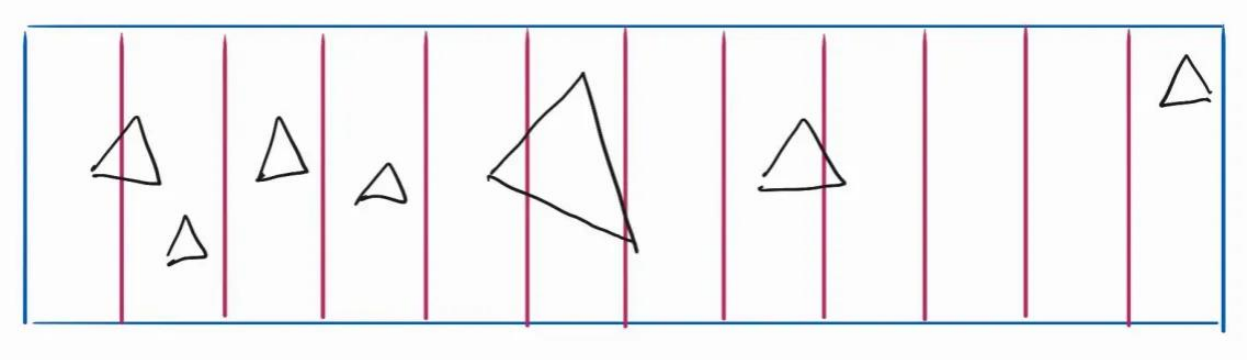

Figure 4. Visualization of Surface Area Heuristic BVH Tree Generation

This is repeated if there are still many primitives on either side after the split. Eleven splits was chosen semi-arbitrarily. It was required to have a substantially larger number than 1 split, because SAH with 1 split behaves the same as SMS.

The primary issue with SAH is that it is more computationally expensive to generate the tree than SMS, but it theoretically should still override it during tree traversal. For a small amount of primitives however, SAH may become overkill.

4.3 Remaining Challenges

SAH is recognized as a more optimal solution when compared to SMS, but this version is not the globally optimal solution. The ideal case would check an infinite number of splits, which would not be computationally possible.

The most optimal size of leaf nodes is also difficult to predict. It is generally better to have a small leaf node size when the primitives are far apart. Otherwise, the surface area of the bounding boxes would be very large. For the scenes tested in the following section, the leaf node size was chosen arbitrarily, but kept constant when comparing the two BVH generation methods. This makes it difficult to truly compare the methods for all cases.

BVH Tree generation must be run before rendering. Therefore, it takes longer than the naïve solution in setup but should theoretically be faster during intersection calculations during runtime.

5.0 Renders and Time Comparisons

5.1 Scene Setup

Three images were tested:

Figure 5. Render of Spheres

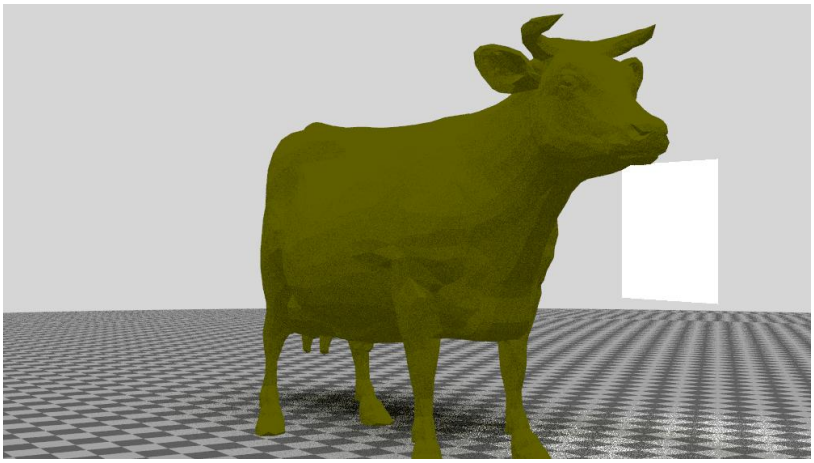

Figure 6. Render of the MIT cow

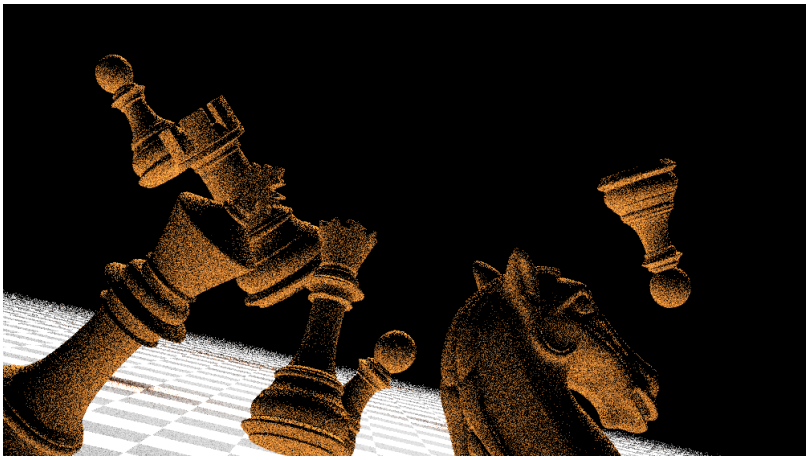

Figure 7. Render of chess pieces

For reproducibility, the tables below describe the setup for each scene. Often, ray tracers will cast multiple rays per pixel in a technique called anti-aliasing. In addition, for colour accuracy, they will often allow rays to bounce, resulting in additional computation and intersection tests. These techniques were not used for the report's tests.

Table 1. Scene parameters for Ray Tracer Sample Renders

	# Primitives	Image Width (px)	Image Height (px)
Spheres	200	1200	675
Cow	5807	1200	675
Chess	278104	800	450

Table 2. Max Primitives per Leaf Node for Tested Scenes

	Leaf Node Size	
	SMS BVH	SAH BVH
Spheres	8	8
Cow	128	128
Chess	256	256

5.2 Results

Table 3. Scene Details for Each Ray-Object Interaction Method

	Rendering Duration (mins)		
	Naive	SMS BVH	SAH BVH
Spheres	37	8.5	1.5
Cow	~6000	32	6
Chess	>30000	243	51

Table 4. Scene Details for Each Ray-Object Interaction Method

	Average Comparisons per ray			Criteria = $10Log_2$(naïve method average comparisons per ray)
	Naive	SMS BVH	SAH BVH	
Spheres	200	11	5	76.43
Cow	5807	43	24	125
Chess	278104	243	137	180.85

Note that the average comparisons per ray for the naïve case is equal to the number of primitives in the scene always.

5.2.1 Time Comparison

Table 4. shows that the SAH BVH acceleration method successfully met the criteria for all scenes. The SMS structure only achieved it for the first two scenes.

Based on the collected data, the scalar of 10 in the criteria is quite overpowering for a small number of primitives, but less significant as the number of primitives grows. It can be assumed that for an even larger number of objects, the criteria with a scalar of 10 would eventually fail to be effective, even for the SAH method. As mentioned, however, this mathematical criterion was chosen semi-arbitrarily. This is because it is visually clear that the average number of ray intersection tests for the BVH methods is fractional when compared to the naïve solution.

The rendering duration of the naïve solution could not be accurately recorded for the chess piece, as seen in Table 3. The testing computer ran for over 21 straight days, then crashed midway losing all the progress of the render. From test logs, it had completed just over 65 percent of the render. As such, the decision was made to not attempt to re-render the chess piece scene with the naïve ray tracer.

The full chess-scene render time for the naïve method can be estimated by assuming that the method progressed linearly; that 30000 minute was exactly 65% of the time needed to fully complete the render. By this process, the full render time would be ~46000 minutes, or around 32 days.

5.2.2 Results Overview

The naïve solution is unrealistic to use for complex scenes (such as the chess scene. It was still incredibly slow for the cow scene. When developers or entertainment companies need to test out scenes before production and post-production, they do not have the time to wait weeks to render a scene, and then make iterations.

Alternatively, it is immediately clear that BVH methods provide a rendering speed-up. The SAH method was able to meet the report's criteria. However, the methods would likely not meet the criteria in an even more complex scene. This is partially a limitation of the criteria itself. Time Complexity is an abstract design used to illustrate how certain algorithms are faster than others as the number of inputs grows to infinity.

While impossible to test, the BVH algorithms would likely continue to be smaller with infinite primitives. A more complex criteria should be chosen instead for future cases, and recommendations will be made in Section 6.1.

The report will still consider exploration successfully meeting the loose criteria, with the consideration that more complex scenes may require a change of criteria. As mentioned in Section 4.2, the SAH algorithm used is not always the universally optimal method, but it still offers a significant and observable speedup for all tested scenes.

6 Conclusions

6.1 Conclusion

The report outlined the design and implementation of ray tracing optimization algorithms for a single-threaded CPU based ray tracer. It is based on a simple static ray tracer that casts out one ray per pixel, and explores two separate Bounding Volume Hierarchy structure generation methods.

Three scenes were tested with the naïve implementation, Spatial Median Split BVH generation, and Surface Area Heuristic BVH generation. Both BVH methods expressed a clear rendering speedup, reducing the number of ray-object intersection tests with a speed up that is observably log-like. These algorithms only modify the number of required ray-intersection computations, and are independent of further mathematical computations for textures, lighting, effects, etc. The SAH method exhibited more efficient results when compared to the SMS generation.

The chess scene was the most complex, with 278104 primitives, the naïve implementation had an average of 278104 intersection calculations per pixel. The SMS method resulted in an average 243, and the SAH method with an average of 137. This is a 99.95% reduction. While the chess scene could not be rendered fully on the naïve solution, it only finished around 65% in 21 days. This illustrates how vital optimization algorithms are, regardless of which method is chosen. Movies, games, and other artistic mediums typically require a high number of primitives, which may be more than the chess scene tested. Using acceleration algorithms may directly reduce costs and optimize production schedules as well.

6.2 Recommendations

The criteria used in the report was loosely mathematical, based on a scalar of the time complexities of the respective algorithms. Rather than using a scalar, a more complex criteria function should be explored, potentially based on the number of primitives in the scene.

GPU multi-threading may further reduce the speed of decreasing rendering duration. While it would not reduce the overall number of computations, multi-threading would allow the ray tracer to compute multiple pixel colors at the same time. This would provide a significant speedup when compared to the sequential CPU-based single threaded processing used in the report.

Computer graphics optimization research has breadth and depth far beyond the scope of this report. There are numerous research papers that describe further optimizations for BVH. An actionable next step could be to modify the splitting method used in BVH construction. The report tested 11 splits in the SAH method, though more splits could be tested, or multiple axis' could be tested.

There may also be ways to reduce the number of primitives by reducing the polygons in meshes. This would further reduce the required number of ray intersection tests for all ray tracing methods.

References

[1] U. o. Utah, "Utah Model Repository," University of Utah, [Online]. Available: https://users.cs.utah.edu/~dejohnso/models/teapot.html. [Accessed 06 05 2025].

[2] "Bounding Volume Hierarchies," UCBerkeley, [Online]. Available: https://cs184.eecs.berkeley.edu/su20/docs/proj3-1-part-2 .

[3] G. f. Geeks, "Time Complexity and Space Complexity," Geeks for Geeks, 05 12 2025. [Online]. Available: https://www.geeksforgeeks.org/time-complexity-and-space-complexity. [Accessed 06 05 2025].

[4] "CUDA C++ Programming Guide," NVIDIA, [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/. [Accessed 05 05 2025].

[5] P. B. a. M. E. a. M. Magno, "The Minimal Bounding Volume Hierarchy," *Vision, Modeling, and Visualization (2010),* 2010.

[6] "Tree Traversal Techniques," Geeks for Geeks, 11 03 2025. [Online]. Available: https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/. [Accessed 05 05 2025].

[7] O. Emil, "5.2.7 Splitting Strategies," TU Wein, [Online]. Available: https://www.iue.tuwien.ac.at/phd/ertl/node92.html. [Accessed 05 05 2025].

[8] "3.1 Balanced Trees," University of Alberta, [Online]. Available: https://webdocs.cs.ualberta.ca/~holte/T26/balanced-trees.html. [Accessed 05 05 2025].

[9] M. P. e. al, "4.3 Bounding Volume Heirarchies," in *Physically Based Rendering, fourth edition: From Theory to Implementation*, The MIT Press, 2018.